

2교시: named volume과 database persistence

수업 목표

- database용 named volume을 생성한다.
- PostgreSQL container에 volume을 mount한다.
- container 교체 후에도 SQL data가 유지되는지 확인한다.

강의 전개

named volume은 Docker가 관리하는 persistent storage다. 학생이 직접 host path를 기억하지 않아도 Docker가 volume lifecycle을 관리한다. database container에서는 container lifecycle과 data lifecycle을 분리하는 것이 핵심이다. 이 교시는 user, database, table, row를 다시 만든 뒤 container를 삭제하고 같은 volume을 새 container에 연결해 data가 유지되는지 확인한다.

이 교시는 설명만 듣고 지나가지 않는다. 명령은 반드시 code block으로 실행하고, 바로 이어서 검증 명령을 실행한다. 정상 출력이 다를 수 있는 부분은 전체 문자열을 외우지 않고 성공 패턴을 기록한다. 실패도 수업 산출물이다. 실패한 명령, 에러 요약, 가설, 다시 확인한 명령을 함께 남긴다.

실습 명령

```
docker volume create paperclip-pg16-data
docker run -d --name paperclip-pg16 -e POSTGRES_PASSWORD=postgres -e
POSTGRES_DB=paperclip -p 15432:5432 -v paperclip-pg16-
data:/var/lib/postgresql/data postgres:16
```

```
docker exec paperclip-pg16 psql -U postgres -d paperclip -c "CREATE TABLE IF
NOT EXISTS notes(id serial PRIMARY KEY, body text); INSERT INTO notes(body)
VALUES ('volume keeps data'); SELECT * FROM notes;"
```

검증 명령

```
docker volume ls | grep paperclip-pg16-data
docker exec paperclip-pg16 psql -U postgres -d paperclip -c "SELECT * FROM
notes;"
```

```
docker stop paperclip-pg16
docker rm paperclip-pg16
docker run -d --name paperclip-pg16-v2 -e POSTGRES_PASSWORD=postgres -e
POSTGRES_DB=paperclip -p 15432:5432 -v paperclip-pg16-
data:/var/lib/postgresql/data postgres:16
docker exec paperclip-pg16-v2 psql -U postgres -d paperclip -c "SELECT * FROM
notes;"
```

실패 드릴과 오해 교정

상황	해석
새 container에서 row가 보인다	volume이 data lifecycle을 보존한 것이다.
새 container에서 초기화 에러	기존 PGDATA와 image major version 또는 mount path를 확인한다.
volume을 지우면 복구 안 됨	volume rm은 data 삭제 명령이다.

Cleanup

```
docker stop paperclip-pg16-v2 || true
docker rm paperclip-pg16-v2 || true
# 실습 데이터를 완전히 지울 때만 실행
# docker volume rm paperclip-pg16-data
```

Cleanup은 비용과 데이터 안전을 동시에 다룬다. container를 지우는 명령과 volume/network/image를 지우는 명령은 의미가 다르다. 특히 volume 삭제는 database data 삭제일 수 있으므로 실습 volume인지 확인한 뒤 실행한다.

Evidence

항목	제출 기준
Volume 이름	paperclip-pg16-data
Persistence 결과	container 교체 전후 SELECT 결과
삭제 판단	volume keep/remove 결정 이유

강의자 설명 포인트

이 실습의 핵심은 명령어 자체가 아니라 경계다. container는 실행 단위이고, volume은 data lifecycle이며, network는 통신 경계다. 학생이 docker run 한 줄을 볼 때 -v, --network, -p를 옵션 목록으로 외

우면 뒤에서 Compose와 Kubernetes로 넘어갈 때 같은 혼란이 반복된다. 그래서 각 옵션을 "무엇을 container 밖으로 분리하는가"라는 질문으로 읽게 한다.

강의 중에는 성공 출력보다 실패 출력의 의미를 더 오래 다룬다. port가 열리지 않은 것은 web server 문제가 아닐 수 있고, DB 접속 실패는 password 문제가 아니라 network boundary 문제일 수 있다. host terminal, container 내부, 같은 Docker network의 client container는 모두 서로 다른 관찰 위치다. 학생이 어디에서 명령을 실행하는지 말로 먼저 설명한 뒤 CLI를 실행하게 한다.

운영 해석

실무에서 database container를 다룰 때 가장 위험한 실수는 cleanup을 단순 정리로만 보는 것이다. container 삭제는 process와 container writable layer를 정리하는 것이고, volume 삭제는 data를 삭제하는 것이다. network 삭제는 통신 경로를 정리하는 것이다. 이 세 가지를 구분하지 않으면 실습은 성공해도 운영 사고를 배운 셈이 된다.

README에는 "실행됐다" 대신 "어떤 data를 남기고 무엇을 삭제했는지"를 써야 한다. Day 2 evidence는 Day 5 Compose에서 volumes와 networks를 읽는 기준이 된다. Compose의 YAML 항목은 갑자기 생긴 문법이 아니라 Day 2에서 손으로 실행한 storage/network 결정을 파일로 옮긴 것이다.

README 기록 예시

```
## Storage/Network Evidence
- Container:
- Volume name:
- Volume target path:
- Network name:
- Host port published? yes/no
- Container DNS check:
- Data survived container replacement? yes/no
- Cleanup decision:
```

다음 연결

다음 교시는 volume 명령을 더 자세히 읽고, bind mount와 named volume의 차이를 비교한다.